

01 / DER EINSTIEG

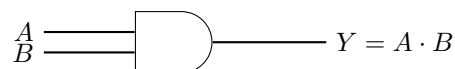
Was macht ein Logikgatter?

Dein Tagesziel: Logische Bedingungen als Gleichung, Wahrheitstabelle und Schaltung darstellen. NOT, AND, OR und XOR aus NAND bauen; De Morgan nachweisen. Außerdem verstehen, warum reale Spannungen zuverlässig als 0 oder 1 gelten.

Ein **Logikgatter** verarbeitet Eingangssignale und erzeugt einen Ausgang. Heute bedeutet 1 „wahr“ und 0 „falsch“. Anders als an Tag 4 addieren wir hier nicht automatisch Stellenwerte: Wir untersuchen zunächst einzelne logische Signale.

Ein Beispiel: Zwei Bedingungen müssen erfüllt sein

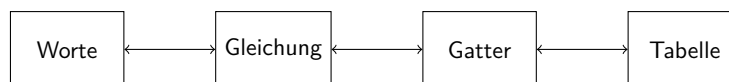
Eine vereinfachte Maschinenfreigabe soll nur aktiv sein, wenn der Startknopf gedrückt *und* die Klappe geschlossen ist. A steht für Start, B für Klappe geschlossen, Y für Freigabe. Das ist AND.



A	B	Y : Freigabe
0	0	0: keine Bedingung erfüllt
0	1	0: Start fehlt
1	0	0: Klappe ist offen
1	1	1: beide Bedingungen erfüllt

Dies ist eine **Wahrheitstabelle**. Sie enthält für zwei Eingänge alle $2^2 = 4$ Kombinationen. Für drei Eingänge braucht sie $2^3 = 8$ Zeilen. Die Eingänge werden wie Binärzahlen aufgezählt: 00, 01, 10, 11. Das Beispiel erklärt eine logische Funktion, keine vollständige Sicherheitssteuerung.

Vier Darstellungen derselben Funktion



Worte sagen, *was* passieren soll. Die Gleichung beschreibt die Bedingung. Die Schaltung verbindet Bauteile. Die Tabelle prüft, ob jede Kombination stimmt.

Die Zeichen richtig lesen

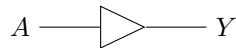
Zeichen	Lesart	Beispiel
$\overline{A}$	NOT A, A negiert	$\overline{0} = 1$
$A \cdot B$ oder AB	A AND B	$1 \cdot 0 = 0$
$A + B$	A OR B	$1 + 1 = 1$ in Boolescher Algebra
$A \oplus B$	A XOR B	$1 \oplus 1 = 0$

Wichtig: Das Plus in einer Booleschen Gleichung bedeutet OR, keine gewöhnliche Addition. Rechenreihenfolge: Klammern, NOT, AND, OR. Ein Überstrich gilt für den gesamten Ausdruck darunter.

Buchbezug: DDCA RISC-V, Abschnitt 1.5, S. 17–20. Abbildungen in diesem Skript sind eigene Zeichnungen.

02 / NACHSCHLAGEN

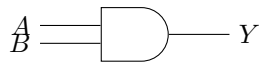
Die Gatter auf einen Blick



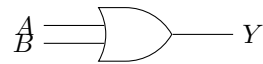
$$\text{BUF} \quad Y = A$$



$$\text{NOT} \quad Y = \overline{A}$$



$$\text{AND} \quad Y = AB$$



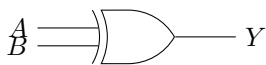
$$\text{OR} \quad Y = A + B$$



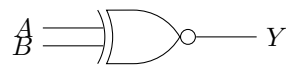
$$\text{NAND} \quad Y = \overline{AB}$$



$$\text{NOR} \quad Y = \overline{A + B}$$



$$\text{XOR} \quad Y = A \oplus B$$



$$\text{XNOR} \quad Y = \overline{A \oplus B}$$

Der kleine Kreis am Ausgang heißt **Inversionsblase**: Er kehrt das Ergebnis um. NAND ist AND mit negiertem Ausgang, nicht „beide Eingänge negieren“.

Die vollständige Tabelle für zwei Eingänge

A	B	AND	OR	XOR	NAND	NOR	XNOR
0	0	0	0	0	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	1

OR: mindestens einer ist 1, auch beide dürfen 1 sein. **XOR**: bei zwei Eingängen genau einer ist 1. **XNOR**: bei zwei Eingängen beide gleich. NOT tauscht 0 und 1; der Buffer gibt seinen Eingang unverändert weiter. Ein Buffer ist logisch wie eine Leitung, kann elektrisch aber eine Last treiben.

Mehr als zwei Eingänge

AND verlangt, dass *alle* Eingänge 1 sind. OR verlangt *mindestens einen*. XOR mit mehreren Eingängen ist 1 bei einer **ungeraden Anzahl** von Einsen. Drei Einsen ergeben daher beim Drei-Eingang-XOR ebenfalls 1.

Verbindung zu Tag 4

Bei der Addition zweier einzelner Bits A und B gilt:

$$\text{Summenbit} = A \oplus B, \quad \text{Übertragsbit} = AB.$$

Für $1 + 1$ entstehen Summenbit 0 und Übertrag 1, also 10_2 . Das ist ein Halbaddierer; ein zusätzlicher eingehender Übertrag gehört noch nicht dazu.

Buchbezug: Abschnitt 1.5, S. 18–20; Halbaddierer als eigene Brücke zu Tag 4. Abbildungen in diesem Skript sind eigene Zeichnungen.

03 / BOOLESCHE ALGEBRA

Gleichungen verstehen und vereinfachen

Die Variablen haben nur die Werte 0 und 1. Die Grundregeln (Axiome) legen NOT, AND und OR fest: etwa $\bar{0} = 1$, $\bar{1} = 0$, $0 \cdot 1 = 0$ und $1 + 1 = 1$. Daraus folgen die Gesetze, mit denen wir Schaltungen vereinfachen.

Gesetz	AND-Form	OR-Form
Neutrales Element	$A \cdot 1 = A$	$A + 0 = A$
Dominierendes Element	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotenz	$AA = A$	$A + A = A$
Komplement	$A\bar{A} = 0$	$A + \bar{A} = 1$
Vertauschen	$AB = BA$	$A + B = B + A$
Gruppieren	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Verteilen	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
Absorption	$A(A + B) = A$	$A + AB = A$
Zusammenfassen	$(A + B)(A + \bar{B}) = A$	$AB + A\bar{B} = A$

Doppelte Negation: $\bar{\bar{A}} = A$. **Dualität:** In einem gültigen Gesetz AND und OR sowie 0 und 1 gleichzeitig vertauschen ergibt wieder ein gültiges Gesetz. Das ist keine Negation der Variablen.

Warum vereinfachen?

$$Y = AB + A\bar{B} = A(B + \bar{B}) = A \cdot 1 = A.$$

Die ursprüngliche Beschreibung verwendet mehrere Operationen; logisch genügt am Ende eine Verbindung von A zu Y. Weniger Gatter können Fläche und Energiebedarf reduzieren. Ob eine konkrete Umsetzung schneller ist, hängt auch von Gattertypen, Lasten und Leitungen ab.

De Morgan: Negation über eine Verknüpfung ziehen

$$\overline{AB} = \bar{A} + \bar{B} \quad \overline{A + B} = \bar{A}\bar{B}$$

Merke: **Verknüpfung wechseln und jeden Eingang negieren.** „Nicht beide“ ist „mindestens einer nicht“. „Keiner von beiden“ ist „beide nicht“. **Falsch** wäre $\overline{AB} = \bar{A}\bar{B}$.

A	B	$\overline{AB}$	$\bar{A} + \bar{B}$	$\overline{A + B}$	$\bar{A}\bar{B}$
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

Jedes Spaltenpaar stimmt in *allen* Zeilen überein: Das beweist hier die Gleichheit vollständig. Eine einzige geprüfte Kombination wäre kein Beweis.

Weitere Buchregel zum Nachschlagen

Das Konsensgesetz lautet $AB + \bar{A}C + BC = AB + \bar{A}C$. Der Term BC ist logisch redundant: Ist $B = C = 1$, deckt je nach A bereits AB oder $\bar{A}C$ diesen Fall ab. Für die ersten NAND-Schaltungen reichen vor allem doppelte Negation, Idempotenz und De Morgan.

Buchbezug: Ergänzend Abschnitt 2.3, S. 58–63; Tabellen 2.1–2.3. Abbildungen in diesem Skript sind eigene Zeichnungen.

04 / NAND ALS BAUSTEIN

Mit einem Gattertyp viele Funktionen bauen

Wir schreiben kurz $N(A, B) = \overline{AB}$. In den folgenden Zeichnungen ist jeder beschriftete NAND-Kasten genau ein Zwei-Eingang-NAND; seine Ausgangsnegation ist bereits im Namen enthalten. Gleich benannte Signale sind verbunden.

NOT aus einem NAND

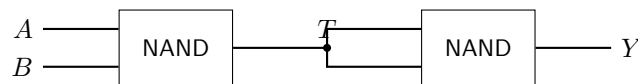
Verbinde *beide* Eingänge mit A . Dann gilt $N(A, A) = \overline{AA} = \overline{A}$.



AND aus zwei NANDs

Erst NAND, dann dessen Ergebnis mit einem weiteren NAND invertieren:

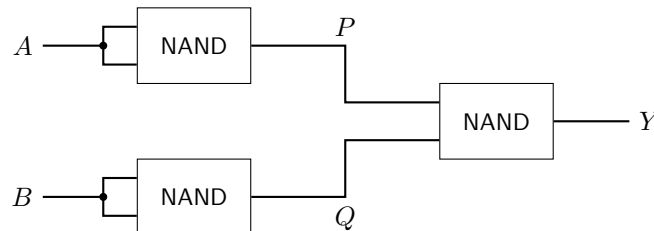
$$T = N(A, B), \quad Y = N(T, T) = \overline{\overline{AB}} = AB.$$



OR aus drei NANDs

Beide Eingänge einzeln negieren und die Ergebnisse NAND-verknüpfen:

$$P = N(A, A), \quad Q = N(B, B), \quad Y = N(P, Q) = \overline{\overline{A}\overline{B}} = A + B.$$



Warum heißt NAND universell?

Aus NAND können wir NOT, AND und OR bauen. Jede Boolesche Funktion mit endlich vielen Eingängen lässt sich aus diesen Operationen darstellen, etwa als OR der AND-Terme für alle Tabellenzeilen mit Ausgang 1. Deshalb genügt NAND allein für jede solche *kombinatorische* Funktion. Daraus folgt nicht, dass jede Funktion nur ein NAND benötigt oder dass NAND immer die beste Umsetzung ist.

Buchbezug: Ableitungen aus 1.5 und 2.3; Bezug Wahrheitstabelle/Gleichung: 2.2 und 2.4. Abbildungen in diesem Skript sind eigene Zeichnungen.

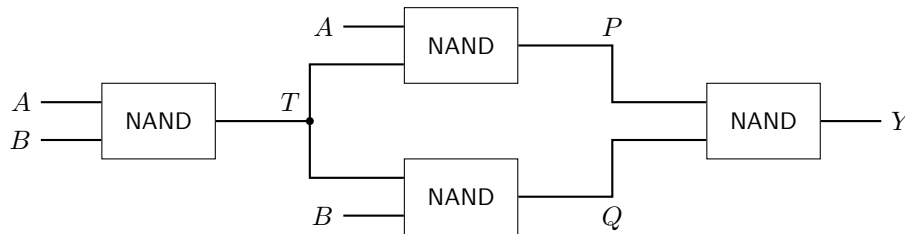
05 / AUFBAU UND NACHWEIS

XOR und De Morgan in Schaltungen

XOR aus vier NANDs

Definiere zuerst Zwischenwerte, statt die ganze Schaltung auf einmal zu lesen:

$$T = N(A, B), \quad P = N(A, T), \quad Q = N(B, T), \quad Y = N(P, Q).$$



A	B	T	P	Q	Y
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	0

Zum Beispiel $A = 0, B = 1$: $T = 1, P = 1, Q = 0$, also $Y = N(1, 0) = 1$. Die Ergebnisspalte entspricht XOR. Das ist ein vollständiger Funktionsnachweis.

De Morgan als Umzeichnen von Gattern



Oben: NAND entspricht OR mit *beiden* negierten Eingängen. Unten: NOR entspricht AND mit *beiden* negierten Eingängen. Eine Ausgangsblase über den Gatterkörper schieben heißt: AND/OR tauschen und an *jedem* Eingang eine Blase einsetzen. Zwei aufeinanderfolgende Negationen heben sich logisch auf.

Zwei Nachweise in deiner Übung

Vergleiche im ersten Versuch $\overline{A}\overline{B}$ mit $\overline{A + B}$, im zweiten $\overline{A + B}$ mit $\overline{A}\overline{B}$. Teste jeweils 00, 01, 10 und 11. Beide Ausgänge müssen in jeder Zeile gleich sein. Ein optionales XOR zwischen den beiden Ausgängen muss dann immer 0 liefern.

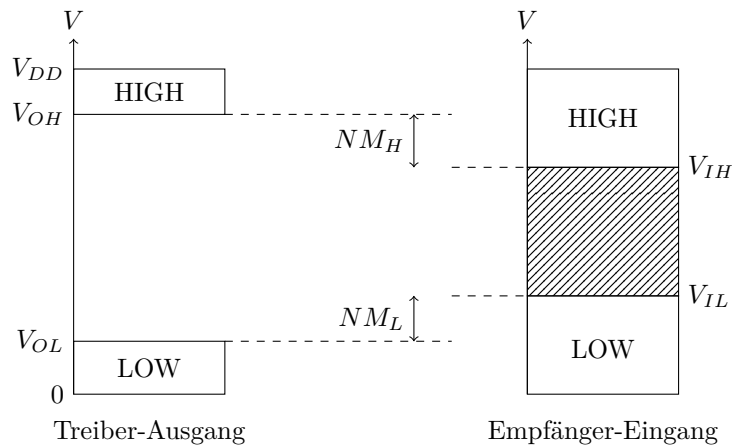
Buchbezug: *De Morgan*: 2.3, S. 61–62; *Bubble Pushing*: 2.5.2; eigene NAND-XOR-Ableitung. Abbildungen in diesem Skript sind eigene Zeichnungen.

Warum 0 und 1 Spannungsbereiche sind

Ein Draht trägt keine aufgedruckte 0 oder 1, sondern eine kontinuierliche Spannung. Ein Gatter muss auch leicht gestörte Spannungen richtig lesen. Deshalb steht nicht nur exakt 0 V für LOW und exakt V_{DD} für HIGH.

Treiber und Empfänger unterscheiden

Der **Treiber** erzeugt eine Ausgangsspannung; der **Empfänger** liest sie. V_{DD} ist die Versorgungsspannung, GND das Bezugspotential 0 V.



Schraffiert: kein garantierter Logikwert. Die Zeichnung ist schematisch und nicht maßstäblich. Sie zeigt keine realen Bauteilkennwerte.

V_{OL}	höchste garantierte LOW-Ausgangsspannung
V_{OH}	niedrigste garantierte HIGH-Ausgangsspannung
V_{IL}	höchste noch als LOW akzeptierte Eingangsspannung
V_{IH}	niedrigste schon als HIGH akzeptierte Eingangsspannung

Zwischen V_{IL} und V_{IH} ist die Interpretation nicht garantiert. Das ist kein dritter normaler Boolescher Wert.

Störabstand: Wie viel Reserve bleibt?

$$NM_L = V_{IL} - V_{OL}$$

$$NM_H = V_{OH} - V_{IH}$$

Eigene Beispielwerte: $V_{DD} = 3,3$ V, $V_{OL} = 0,2$ V, $V_{IL} = 0,8$ V, $V_{IH} = 2,0$ V, $V_{OH} = 2,8$ V. Dann sind $NM_L = 0,6$ V und $NM_H = 0,8$ V. Ein Anstieg um 0,5 V auf dem schlechtesten LOW bleibt LOW; ein Abfall um 0,9 V auf dem schlechtesten HIGH verlässt den garantierten HIGH-Bereich.

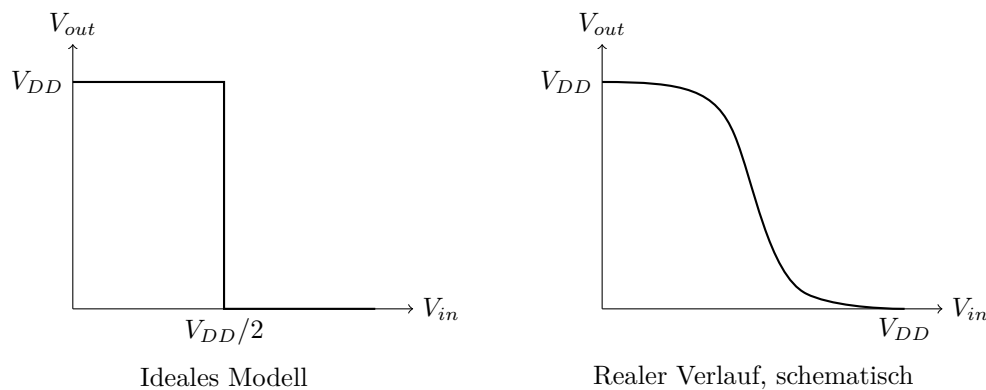
Buchbezug: Abschnitte 1.6.1–1.6.3, S. 20–22; eigene Grafik nach dem Konzept von Abb. 1.23. Abbildungen in diesem Skript sind eigene Zeichnungen.

07 / VOM ANALOGEN ZUM DIGITALEN

Kennlinie und zuverlässige Verknüpfung

Ein Inverter ist elektrisch nicht ideal

Die DC-Übertragungskennlinie zeigt V_{out} als Funktion von V_{in} bei langsam veränderter Eingabe. Ein idealer Inverter würde abrupt umschalten. Ein realer Inverter besitzt einen kontinuierlichen Übergang.



Bei kleiner Eingangsspannung ist der Ausgang hoch, bei großer niedrig. Der reale Übergang muss nicht genau bei $V_{DD}/2$ liegen. Das Buch bestimmt die Eingangsschwellen an den zwei Stellen, an denen

$$\frac{dV_{out}}{dV_{in}} = -1.$$

Das heißt lokal: Eine kleine Erhöhung am Eingang bewirkt eine gleich große Absenkung am Ausgang. Die Ableitung musst du heute nicht selbst berechnen; wichtig ist, dass Logikpegel aus elektrischen Eigenschaften entstehen.

Static Discipline: Gültig hinein, gültig heraus

Bei gültigen, stabilen Eingangssignalen soll jedes Gatter nach seiner Einschwingzeit gültige Ausgangssignale liefern. So lassen sich viele Gatter verbinden, ohne jede Kombination vollständig analog analysieren zu müssen. Während eines Pegelwechsels durchläuft eine reale Spannung auch Zwischenwerte; die statische Regel beschreibt nicht das gesamte zeitliche Verhalten.

Passen zwei Gatter elektrisch zusammen?

Prüfe die garantierten Ausgangspegel des Treibers gegen die Eingangsanforderungen des Empfängers. Für positive Störreserve benötigen wir

$$V_{OL} < V_{IL}, \quad V_{OH} > V_{IH}.$$

Zusätzlich müssen die zulässigen Eingangsspannungen eingehalten werden. Ein 5-V-Ausgang darf nicht automatisch an einen 3,3-V-Eingang angeschlossen werden: Auch ein logisch eindeutiges HIGH kann elektrisch zu hoch sein. Die konkreten Grenzwerte hängen von der Bauteilfamilie ab.

Was Logisim zeigt und was es vereinfacht

Eine Logiksimulation hilft dir, Wahrheitstabellen und Verbindungen zu prüfen. Sie ersetzt keine Untersuchung realer Spannungsgrenzen, Ströme und Lasten. Ein logisch korrekter NAND-Aufbau ist daher der erste Schritt zur Hardware; die elektrische Umsetzung kommt später dazu.

Buchbezug: Abschnitte 1.6.4–1.6.5, S. 22–24; eigene Kennlinien nach Abb. 1.25. Abbildungen in diesem Skript sind eigene Zeichnungen.

Vom Lesen zur geprüften Schaltung

So liest und prüfst du einen Aufbau

1. Benenne Eingänge (A, B), Zwischenwerte (T, P, Q) und Ausgang (Y).
2. Notiere für jedes Gatter seine Gleichung. Gehe von links nach rechts vor.
3. Erstelle die vollständige Wahrheitstabelle mit Zwischenwerten.
4. Baue die Schaltung und teste jede Zeile, nicht nur einen Beispielszustand.
5. Vergleiche mit der gewünschten Funktion und speichere den Aufbau.

Ein Verbindungspunkt zeigt eine elektrische Verbindung. Sich kreuzende Linien ohne Punkt sind in den hier verwendeten Schaltplänen nicht verbunden. Ein Ausgang darf mehrere Eingänge speisen; zwei Ausgänge direkt zu verbinden ist keine Methode, AND oder OR zu bilden.

Praktischer Arbeitsauftrag für Tag 5

Der Lernplan sieht Logisim-Evolution vor. Lege ein Projekt mit getrennten Teilschaltungen an. Stelle die Pins auf einzelne Bits und die NANDs auf zwei Eingänge. Baue die folgenden Funktionen ausschließlich aus NAND:

Funktion	NAND-Anzahl dieses Aufbaus	Prüfung
NOT	1	beide Eingabewerte
AND	2	00, 01, 10, 11
OR	3	00, 01, 10, 11
XOR	4	00, 01, 10, 11
NOR (zusätzlicher fünfter Aufbau)	4	00, 01, 10, 11

Für NOR invertierst du den NAND-basierten OR-Ausgang mit einem weiteren NAND. Damit erhältst du die fünf NAND-Schaltungen der Erfolgskontrolle. Der Plan nennt vier Zielfunktionen, fordert aber fünf Schaltungen; NOR ergänzt hier sinnvoll den fünften Aufbau.

Baue außerdem die beiden De-Morgan-Vergleiche von Seite 5. Hier dürfen zum Vergleichen auch die normalen NOT-, AND- und OR-Symbole verwendet werden. Halte für beide Vergleiche alle vier Ausgangspaare fest und speichere dein Projekt als `tag05_nand.circ` in diesem Ordner. Dieses PDF ist die Lernunterlage; es enthält noch keine fertig gebaute Logisim-Datei.

Selbstkontrolle mit kurzen Antworten

Warum ist OR nicht XOR? OR akzeptiert auch zwei Einsen; Zwei-Eingang-XOR nicht.

Warum funktioniert NOT aus NAND? $AA = A$, daher $N(A, A) = \overline{A}$.

Warum reicht NAND? NOT, AND und OR lassen sich daraus zusammensetzen.

Warum alle Tabellenzeilen? Nur so ist die Funktion für jede Eingabe geprüft.

Warum Spannungsbereiche? Reale Signale sind kontinuierlich und gestört.

Warum nicht nur auswendig lernen? Du solltest jeweils zwischen Worten, Gleichung, Tabelle und Schaltung wechseln können.

Lesestellen im vorgesehenen Buch

Harris & Harris: Digital Design and Computer Architecture, RISC-V Edition. Abschnitt 1.5 (Buch S. 17–20) erklärt die Gatter; 1.6 (S. 20–24) erklärt Pegel, Störabstände, Kennlinien und die statische Regel. **Ergänzung zum Lernplan:** Die Booleschen Axiome, Gesetze und De Morgan stehen in 2.3 (S. 58–63), nicht in 1.6. Für die Schaltungsdarstellung und Bubble Pushing siehe 2.4–2.5.2 (S. 64–71). Alle Zeichnungen und didaktischen Beispiele dieser Zusammenfassung wurden neu für deine Übung erstellt. Sie sind keine übernommenen Buchabbildungen.